# A data factory (DataOps) for statistical classification task level, data and application logic modeling

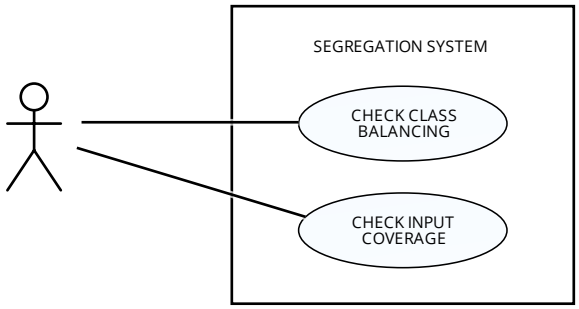
**Mario G.C.A. Cimino, Dept. Information Engineering, University of Pisa**

## TASK MODELING

Task level modeling concerns mainly human tasks. Often a human task is a use case, but sometimes a fragment of workflow with tied human tasks is a unique use case. Use **Visual Paradigm** for use cases, detailed use cases, class diagrams, but for mockups use **Diagrams.net**

For each use case, **add an analysis Trello card, with title: "TASK <name of the task>", description: "use case, detailed use case, mockup".**

First step: for each system, determine its use cases.




For each human task, design a user interface (mockup) suitable for the decision-making task; design a detailed use case for both the interaction and for the decision logic.

For mockups use **Diagrams.net** see the file "From performance to productivity evaluation" for starting examples of mockups.

It follows a textual description of the most important human task. It is strongly recommended to model a standard basic factory in the AS-IS version, and to focus on improvements in the To-BE process.

*Check class balancing*: to check whether classes are balanced or not, on the basis of a balancing tolerance parameter; consider a **bar chart** with the number of instances for each class; e.g. 5% of balancing tolerance means that the numbers of instances can be different for at most 5% with respect to their average. Consider that a number of sessions to collect is set in the development phase, as well as a number of sessions needed for training.

*Check input coverage*: to check whether the input space of the network is definitively covered for each feature. It means that by collecting other sessions, the distribution does not change. Consider a **radar scatter** (for continuous features) or **radar bubble** (for discrete features) chart.
https://tinyurl.com/radar-scatter
https://tinyurl.com/radar-bubble

*Machine learning sets:* it is divided into training set, validation set, and test set, extracted as percentages of the total set.

*Training report*: show the training error plot (MSE), calculated on the training set, against the number of iterations, up to a maximum no. of iterations.

Let us assume a simple **feedforward neural networks** (e.g. fine tuning of the classification layer), with two hyperparameters: no. of layers and no. of neurons per layer. As a **grid space**, for each hyperparameter, set the minimum, the variation step, and the maximum.

*Validation report*: carry out a grid search, by training different networks, one for each point of the grid space. Finally, show a table with the top 5 classifiers, ordered by the final MSE on the validation set. In the table, for each classifier, show its validation error, training error, no. of layers, no. of neurons. Find the best network, i.e., the lowest validation error, for which the difference between validation and training error is under the overfitting tolerance. Find the second-

best network. If the validation error between the two network is very similar, i.e. the difference is one order of magnitude w.r.t. their error, then select the network with the lowest complexity

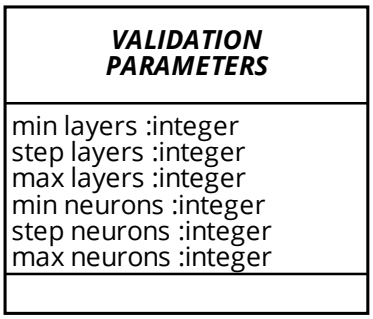
*Testing report*: show the winner network (that has been selected from validation): its validation error and test error. Check whether the difference between validation and testing error is under the generalization tolerance.

*Evaluation report*: show a table having, for each session, a pair of labels (expert and classifier); each difference of label in a pair is an error of the classifier. Check whether the total no. of errors and the maximum no. of consecutive errors are both below the n. of errors threshold and the max no. of consecutive errors threshold.

## DATA MODELING

For each Process, **add an analysis Trello card, with title: "DATA MODEL of <name of the process>", description: "UML class diagrams of data objects and data storages".**

For each data object, and for each data storage, develop with Signavio an UML class with the related variables/parameters.




It follows a list of requirements of the various script tasks.

*mark missing samples*: some samples can be lost in records generated by client-side devices, e.g. for power loss, electronic noise, redundancy (the sample/record is static, it is sent only when its value changes), and so on. If missing samples are detected, a mark is used as a placeholder, and the number of missing samples is calculated. If the total number of missing samples is higher than a max missing samples, then the raw session is discarded.

*Correct missing samples*: the missing samples are interpolated (for time series) or copied from the last sample/record (for static records).

*Correct absolute outliers*: due to electronic noise or power loss, some value can take on a value that is not possible (e.g. heart rate greater than 300 bpm). When the value is greater/less than an upper/lower bound, it is corrected to the upper/lower bound.

*Extract features*: Example of features are provided in the data structure. However, the set of features to extract depends on the data complexity. Assume that the preparation system potentially provides a large number of predefined features. Then, a human configures which subset of features will be extracted.

*Set average hyperparameters*: the value of each hyperparameter is set to (max+min)/2

*Set hyperparams*: the next combination of hyperparameters in the grid search is set.

## APPLICATION LOGIC MODELING

For each Process, **add an analysis Trello card, with title: "APPLICATION LOGIC of <name of the process>", description: "UML class diagrams of the application logic (script task, human task), Workflow realization (sequence diagram)".**

In addition to data objects and data storage, other classes are needed to model the tasks and their workflow. Some paradigms to use:

1) MODEL-VIEW-CONTROLLER, for each user task with a plot, create a separate class for generating the png plot (view) and another for generating the data needed for the plot (model); accompanying input/output information is provided via separated json files;
2) JSON BASED I/O, one member of the team can develop a class for taking and sending

messages, in json format, including the input validation; this class can be delivered to the other members and customized;

3) STORAGE CONTROLLER, one member of the team can develop a class for managing data store, this class can be delivered to the other members and customized;
4) OTHER CONTROLLERS (ORCHESTRATORS), for simple workflow, e.g. ingestion, preparation, evaluation, create one orchestrator class, for managing the flow of work. For complex orchestration, e.g. development, create a hierarchy, i.e. three orchestrator classes (training, validation and testing) and a top orchestrator class.
5) STOP&GO INTERACTION; for the purpose of development of the application logic, the interactive nature of the python applications is simplified according to a "stop&go" pattern. For example, after the generation of the class balancing as a png file: (i) the application ends; (ii) the user opens the balancing plot (using an image viewer), opens the accompanying information in a related json file; (iii) the user provides the decision about data balancing in a json file; (iv) the user starts the application; (v) the controller continues according to the decision.
6) TESTING and SIMULATED INTERACTION; for the purpose of the testing of the factory (not the testing of the classifier!) the orchestrators have an additional service flag. If the service flag is on, the stop&go" interaction is simplified: the user decision is statistically generated on-the-fly, and then the application controller does not stop. Moreover, the service flag enables a generation of timestamps, which are sent to the service class (see below).
7) SERVICE CLASS; a service class is developed for simulating client-side systems and managing testing functions. It takes random records from the local csv files; it sends json messages to the ingestion system; it receives json messages in place of client-side and messaging systems; for the purpose of testing log, it generates the initial timestamp, takes the intermediate and final timestamps from the other systems, generates a csv timestamp log. The testing log will be processed via a spreadsheet software for generating the responsiveness / elasticity plots of development and execution phases.
8) WORKFLOW REALIZATION, develop a sequence diagram for realizing the interaction between all classes (objects) for each process.

** END OF THE ANALYSIS WORKFLOW **

**DESIGN WORKFLOW**:

For each Process, add a design Trello card, with title: "**DATA & APPLICATION LOGIC of <name of the process>**", description: "UML class diagrams of the data and application logic (script task, human task), Workflow realization (sequence diagram)". The class diagram is generated from the code using Pyreverse:
https://pylint.readthedocs.io/en/latest/pyreverse.html

**IMPLEMENTATION WORKFLOW:**

For each Process, add a design Trello card, with title: "**DATA and APPLICATION LOGIC of <name of the process>**", description: "Python implementation on the Pycharm IDE; Deployment diagram".

**TESTING WORKFLOW:**

List of cards

1) Title: RESPONSIVENESS-ELASTICITY of the production phase
   Description: experiments for producing logs, generation of the plot via spreadsheet, for the production workflow; discussion of results.

2) Title: AUTOMATION RESPONSIVENESS-ELASTICITY of the development phase
   Description: experiments for producing logs, generation of the plot via

spreadsheet, for the execution workflow; discussion of results.

3) Title: AUTOMATION
   Description: calculation of the cognitive cost of each use case

4) Title: RESILIENCY
   Description: calculation of the non-resiliency score of each system

5) Title: INTEROPERABILIY
   Description: calculation of the non-interoperability score of each system